

ESP32

Descripcion:

- Crear la Rama esp32/test-touch-capacitivo
 - Instalar extension ESP-IDF- v1.8.1 en VS Code
 - Instalar ESP-IDF v5.3.5 Manualmente
 - Resolucion del Bloqueo Windows Defender sobre Toolchain
 - Instalar Driver CP2102 para reconocimiento del puerto de la ESP32
 - Instalacion de PlatformIO y configurar proyecto
 - Test del sensor Capacitivo
 - Implementacion del Deep Sleep
-

1) INSTALACION DEL ENTORNO ESP-IDF

Extension VS Code

La version 2.1.0 de la extension ESP-IDF de Espressif tenia un bug: el comando 'ESP-IDF: Open Get Started Walkthrough' no abria nada. Se soluciono instalando la version 1.8.1 que tiene el wizard funcional.

```
code --install-extension espressif.esp-idf-extension@1.8.1
```

INSTALACION MANUAL DEL TOOLCHAIN

El wizard fallo repetidamente por errores de descarga del ZIP desde GitHub (archivos corruptos, cancelaciones accidentales). Se opto por instalacion manual:

- 1 Descargar esp-idf-v5.3.5.zip desde GitHub Releases (~200 MB)**
- 2 Extraer: Expand-Archive -Path ...zip
-DestinationPath C:\Users\pulid\esp\v5.3.5\esp-idf**
- 3 Navegar a la carpeta: cd ...\\esp-idf-v5.3.5**
- 4 Ejecutar como admin: .\\install.ps1 esp32**
- 5 Activar entorno: .\\export.ps1 6 Verificar: python ...\\tools\\idf.py --version →
ESP-IDF v5.3.5-dirty**

Problema: Windows Defender bloqueaba el toolchain

Al ejecutar install.ps1 aparecio el error: [WinError 4551] Una directiva de Control de aplicaciones bloqueo este archivo. Se resolvio ejecutando PowerShell como administrador y agregando exclusiones de Windows Defender:

```
Add-MpPreference -ExclusionPath "C:\Users\pulid\espressif"  
Add-MpPreference -ExclusionPath "C:\Users\pulid\esp"
```

2) Driver CP2102

El ESP32 utilizado tiene chip CP2102 para la comunicacion USB-UART. Windows no lo reconocia (status Error en Administrador de dispositivos). Se instalo el driver manualmente con pnputil al no encontrarse el .exe en el paquete descargado:

```
pnputil /add-driver  
C:\Users\pulid\Downloads\CP210x_Universal_Windows_Driver\silabser.inf /install
```

3) PlatformIO

Se instalo la extension PlatformIO IDE en VS Code. Se decidio usar PlatformIO con framework Arduino por simplicidad, ya que el codigo del cubrevolante usa las funciones nativas de Arduino (touchRead, Serial) y no requiere ESP-IDF puro

Archivo platformio.ini del proyecto:

```
[env:esp32dev] platform = espressif32 board = esp32dev framework = arduino
monitor_speed = 115200 upload_port = COM6 monitor_port = COM6
```

Estructura del proyecto:

```
esp32/esp32-prueba/esp32_smic/ src/main.cpp <- codigo principal platformio.ini <-
configuracion include/ <- headers lib/ <- librerias locales
```

Comando de upload utilizado:

```
~\platformio\env\Scripts\pio.exe run --target upload
```

4) Sensor capacitivo

Se realizo el primer test del sensor touch capacitivo del ESP32 conectando un cable al GPIO4. Este pin es uno de los 10 pines touch capacitivos del ESP32 y sera el que detecte el contacto de la mano en el cubrevolante.

Codigo utilizado (main.cpp):

```
#include #define PIN_TOUCH 4 void setup() { Serial.begin(115200); Serial.println("Test
touch capacitivo"); } void loop() { int valor = touchRead(PIN_TOUCH); if (valor < 20) {
Serial.println(String(valor) + " -> TOCANDO"); } else { Serial.println(String(valor) + " -> SIN
CONTACTO"); } delay(200); }
```

5) Deep Sleep

Se implemento el ciclo de deep sleep del SmicCubrevolante. En lugar del loop() continuo, el ESP32 ahora se despierta, lee el sensor, reporta el valor y vuelve a dormir. Esto reduce el consumo de ~100mA constante a ~5mA promedio, extendiendo la autonomia de la bateria de 26 horas a mas de 520 horas.

Codigo con deep sleep (main.cpp):

```
#include #include "esp_sleep.h" #define PIN_TOUCH 4 #define UMBRAL_TOUCH 20  
#define SLEEP_US 500000 // 500ms void setup() { Serial.begin(115200); delay(100); int  
valor = touchRead(PIN_TOUCH); bool tocando = valor < UMBRAL_TOUCH;  
Serial.print("Valor: "); Serial.println(tocando ? " -> TOCANDO" : " -> SIN CONTACTO");  
esp_sleep_enable_timer_wakeup(SLEEP_US); Serial.flush(); esp_deep_sleep_start();  
} void loop() { // No se usa - deep sleep reinicia desde setup() }
```

Salida del monitor serial:

**rst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT) Valor: 101 -> SIN
CONTACTO Entrando a deep sleep... rst:0x5 (DEEPSLEEP_RESET),boot:0x13
(SPI_FAST_FLASH_BOOT) Valor: 101 -> SIN CONTACTO Entrando a deep sleep...**

**rst:0x5 (DEEPSLEEP_RESET) confirma que el ESP32 esta despertando desde deep
sleep correctamente.**